

01_cleaning.ipynb

Google Research – Web Traffic Time Series Forecasting

Data Loading, Cleaning, Sampling, Preprocessing

This notebook performs the initial steps of the pipeline:

- Load the raw Kaggle dataset
- Understand the structure
- Handle missing values
- Apply log-transform
- Randomly sample a subset of pages for modeling
- Visualize a few time series
- Save cleaned data for the next notebook

This dataset contains daily Wikipedia page views for ~145,000 pages from 2015–2016. Our goal is to forecast the next 60 days of views.

Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random

# display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.max_rows", 50)
```

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load the Kaggle Dataset

The dataset contains:

- A `train_1.csv` file with page titles as rows and dates as columns
- Daily page views for each article

Load the file using the full path

```
file_path = "/content/drive/MyDrive/CSC480/Final Project/train_1.csv"

df = pd.read_csv(file_path)
df.head()
```

Understanding the Dataset Structure

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	2015-07-10	2015-07-11	2015-07-12	2015-07-13	2015-07-14
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	13.0	14.0	9.0	9.0	22.0	26.0	24.0	19.0	10.0	14.0	15.0
• Each row = one Wikipedia page															
• First column: Page	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	18.0	11.0	13.0	22.0	11.0	10.0	4.0	41.0	65.0	57.0	38.0
• Remaining columns: dates	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	1.0	0.0	4.0	0.0	3.0	4.0	4.0	1.0	1.0	1.0	6.0
• Values: number of page views	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	94.0	4.0	26.0	14.0	9.0	11.0	16.0	16.0	11.0	23.0	145.0
3															

```
print("Shape:", df.shape)
df.info()

Shape: (145063, 551)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145063 entries, 0 to 145062
Columns: 551 entries, Page to 2016-12-31
dtypes: float64(550), object(1)
memory usage: 609.8+ MB
```

Extract Metadata (Language, Access Type, Agent Type)

Each page name in the dataset contains embedded metadata such as:

- Language (e.g., en, de, fr, zh)
- Access type (desktop, mobile, all-access)
- Agent type (user or spider)

We will extract these fields into separate columns for use in modeling.

```
def extract_language(page):
    try:
        return page.split(".")[0]
    except:
        return "unknown"

def extract_access(page):
    if "desktop" in page:
        return "desktop"
    elif "mobile" in page:
        return "mobile"
    else:
        return "unknown"

def extract_agent(page):
    if "spider" in page:
        return "spider"
    else:
        return "user"

df["language"] = df["Page"].apply(extract_language)
df["access"] = df["Page"].apply(extract_access)
df["agent"] = df["Page"].apply(extract_agent)

df[["Page", "language", "access", "agent"]].head()
```

	Page	language	access	agent
0	2NE1_zh.wikipedia.org_all-access_spider	2NE1_zh	unknown	spider
1	2PM_zh.wikipedia.org_all-access_spider	2PM_zh	unknown	spider
2	3C_zh.wikipedia.org_all-access_spider	3C_zh	unknown	spider
3	4minute_zh.wikipedia.org_all-access_spider	4minute_zh	unknown	spider
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_s...	52_Hz_I_Love_You_zh	unknown	spider

Random Sampling

The full dataset (145k pages × 550 days) is too large for fast modeling.

We randomly sample **500 pages** which is enough for good training results.

```
SAMPLE_SIZE = 500
df_sample = df.sample(SAMPLE_SIZE, random_state=42).reset_index(drop=True)
```

df_sample.head()

		Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	2015-07-10	2015-07-11	2015-07-12	2015-07-13
0	Phabricator/Project_management_www.mediawiki.o...		6.0	6.0	4.0	6.0	8.0	6.0	4.0	0.0	2.0	3.0	2.0	3.0	3.0
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...		242.0	271.0	309.0	227.0	321.0	311.0	242.0	236.0	243.0	266.0	381.0	347.0	336.0
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...		3.0	19.0	19.0	30.0	21.0	24.0	17.0	178.0	40.0	1.0	3.0	2.0	4.0
3	Érythrée_fr.wikipedia.org_desktop_all-agents		672.0	513.0	774.0	1164.0	546.0	755.0	555.0	494.0	4801.0	4514.0	1101.0	1113.0	747.0
4	Metallica_es.wikipedia.org_all-access_all-agents		1534.0	1644.0	1704.0	1569.0	1534.0	1577.0	1608.0	1731.0	1919.0	1628.0	1640.0	1646.0	1635.0
5 rows x 554 columns															

Handling Missing Values

We fill missing entries using:

- Linear interpolation across days
- Fill remaining NAs with 0

```
date_cols = df.columns[1:-3] # all date columns
df_sample[date_cols] = df_sample[date_cols].interpolate(axis=1)
df_sample[date_cols] = df_sample[date_cols].fillna(0)

df_sample.head()
```

		Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	2015-07-10	2015-07-11	2015-07-12	2015-07-13
0	Phabricator/Project_management_www.mediawiki.o...		6.0	6.0	4.0	6.0	8.0	6.0	4.0	0.0	2.0	3.0	2.0	3.0	3.0
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...		242.0	271.0	309.0	227.0	321.0	311.0	242.0	236.0	243.0	266.0	381.0	347.0	336.0
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...		3.0	19.0	19.0	30.0	21.0	24.0	17.0	178.0	40.0	1.0	3.0	2.0	4.0
3	Érythrée_fr.wikipedia.org_desktop_all-agents		672.0	513.0	774.0	1164.0	546.0	755.0	555.0	494.0	4801.0	4514.0	1101.0	1113.0	747.0
4	Metallica_es.wikipedia.org_all-access_all-agents		1534.0	1644.0	1704.0	1569.0	1534.0	1577.0	1608.0	1731.0	1919.0	1628.0	1640.0	1646.0	1635.0
5 rows x 554 columns															

Log(1 + x) Transform

Page views vary wildly (0 → millions).
A log-transform stabilizes variance and improves ML model performance.

```
df_log = df_sample.copy()
df_log[date_cols] = np.log1p(df_log[date_cols])
df_log.head()
```

		Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	2015-07-10
0	Phabricator/Project_management_www.mediawiki.o...		1.945910	1.945910	1.609438	1.945910	2.197225	1.945910	1.609438	0.000000	1.098612	1.386294
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...		5.493061	5.605802	5.736572	5.429346	5.774552	5.743003	5.493061	5.468060	5.497168	5.587249
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...		1.386294	2.995732	2.995732	3.433987	3.091042	3.218876	2.890372	5.187386	3.713572	0.693147
3	Érythrée_fr.wikipedia.org_desktop_all-agents		6.511745	6.242223	6.652863	7.060476	6.304449	6.628041	6.320768	6.204558	8.476788	8.415160
4	Metallica_es.wikipedia.org_all-access_all-agents		7.336286	7.405496	7.441320	7.358831	7.336286	7.363914	7.383368	7.457032	7.560080	7.395722
5 rows x 554 columns												

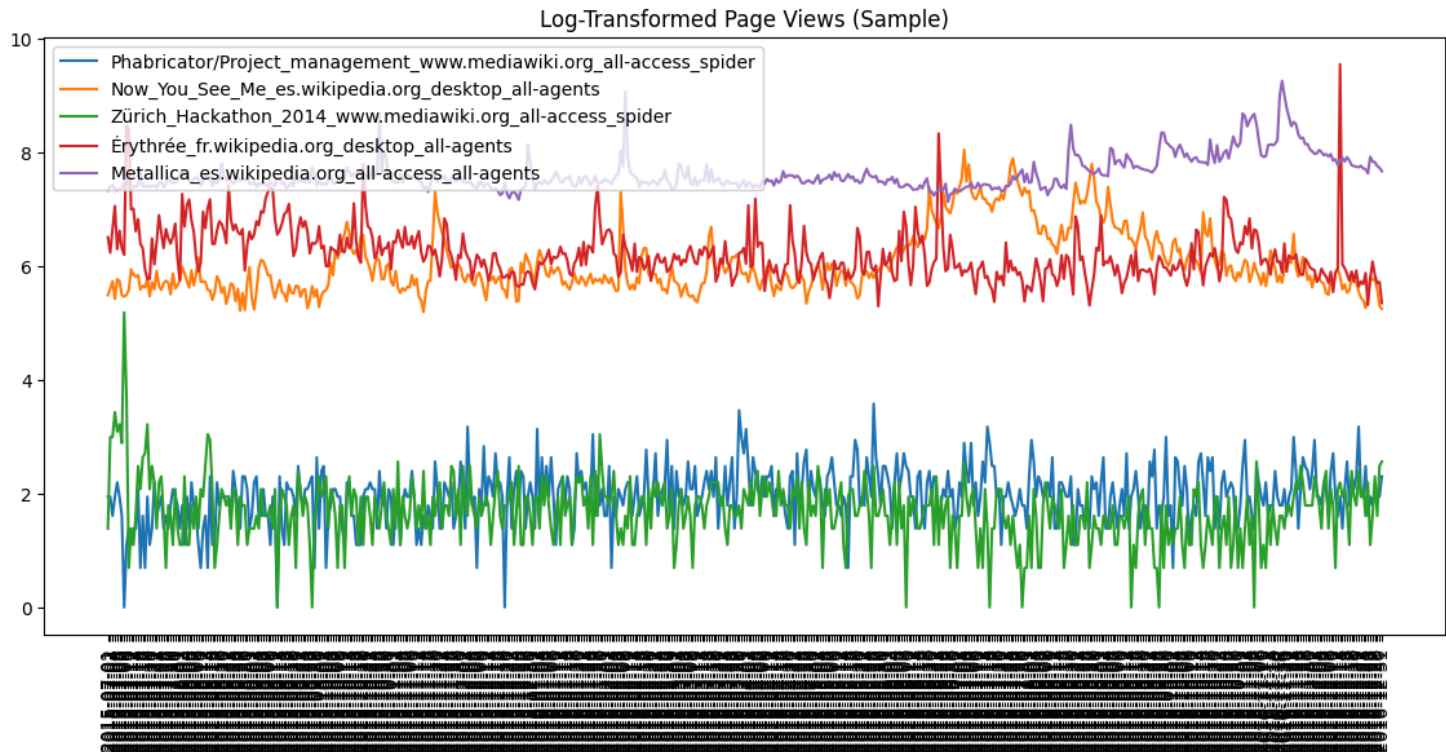
Visualizing Sample Time Series

Plot a few transformed series to understand behavior.

```
plt.figure(figsize=(14, 6))

for i in range(5):
    row = df_log.iloc[i]
    plt.plot(date_cols, row[date_cols], label=row["Page"])

plt.xticks(rotation=90)
plt.legend()
plt.title("Log-Transformed Page Views (Sample)")
plt.show()
```



✓ Save Cleaned Sampled Data

We save the cleaned + log-transformed dataset for the next notebook feature_engineering.

```
output_path = "/content/drive/MyDrive/CSC480/Final Project/cleaned_sample.csv"
df_log.to_csv(output_path, index=False)

print("Saved cleaned dataset to:", output_path)
```

Saved cleaned dataset to: /content/drive/MyDrive/CSC480/Final Project/cleaned_sample.csv

